

Rolling-Origin Cross-Validation

2026-04-17

Introduction

Standard k -fold cross-validation shuffles observations across folds, which violates time ordering and leaks future information into the training data. For time series, this invalidates predictive performance estimates entirely. Rolling-origin cross-validation (also called time-series CV or walk-forward validation) preserves temporal ordering by training up to a cutoff and forecasting the subsequent horizon, then sliding the cutoff forward and repeating. It is the only valid CV scheme for time-series forecast evaluation.

Prerequisites

A working understanding of cross-validation, time-series forecasting, and the difference between in-sample fit and out-of-sample predictive performance.

Theory

For a series of length T , choose an initial training-window size t_0 , a forecast horizon h , and (optionally) a step size for cutoff advancement. For each cutoff $t = t_0, t_0 + s, \dots, T - h$:

1. Train on $\{y_1, \dots, y_t\}$.
2. Forecast $\hat{y}_{t+1}, \dots, \hat{y}_{t+h}$.
3. Compute the error against observed $y_{t+1}, \dots, y_{t+h}$.

Average errors across cutoffs for each horizon $1, 2, \dots, h$ to obtain horizon-specific accuracy estimates. Two windowing variants exist: expanding window (training set grows) and rolling window (fixed-size training set slides forward). Expanding is more efficient when the underlying process is stable; rolling is more robust to regime changes.

Assumptions

Sufficient history that the initial training window estimates the model adequately; the model can be refit at each cutoff (or parameters updated efficiently for computational savings).

R Implementation

```
library(forecast)

x <- AirPassengers
h <- 12

# tsCV: rolling-origin with one-step expanding window
f_ets <- function(y, h) forecast(ets(y), h = h)$mean
err_ets <- tsCV(x, f_ets, h = h)

# RMSE by horizon
sqrt(colMeans(err_ets^2, na.rm = TRUE))
```

```
# Compare ARIMA
f_arima <- function(y, h) forecast(auto.arima(y), h = h)$mean
err_arima <- tsCV(x, f_arima, h = h)
sqrt(colMeans(err_arima^2, na.rm = TRUE))
```

Output & Results

`tsCV()` returns a matrix of forecast errors with one row per cutoff and one column per horizon. Column-wise summaries give horizon-specific RMSE, MAE, or other metrics. Comparing matrices across models identifies where each performs best.

Interpretation

A reporting sentence: “Rolling-origin CV with one-step expanding window and forecast horizon 12 months showed ARIMA outperforming ETS at short horizons (1–3 months: RMSE 22 vs 28) but ETS was more accurate at longer horizons (10–12 months: RMSE 38 vs 45). The horizon-specific differences justify reporting CV-RMSE per horizon rather than a single averaged metric.” Report horizon-specific errors; collapsing them hides the structure that informs model choice.

Practical Tips

- `forecast::tsCV()` implements expanding-window CV with one-step cutoff advancement; for rolling fixed-window, write a small loop.
- For computational cost, increase the initial window and skip cutoffs (advance by more than one observation per step); this trades estimation precision for speed.
- Always report horizon-specific errors; a single averaged RMSE hides the structure that determines which model wins at which horizon.
- Refitting the model at every cutoff is expensive; for ARIMA, fixed-parameter updates (`forecast(Arima(y, model = fit), h = h)`) are faster but ignore changing dynamics.
- `fable::stretch_tsibble()` provides tidy rolling-CV splits that integrate with the modern tidyverts forecasting workflow.
- For nested or hierarchical forecasts, run rolling-origin CV at the bottom level and aggregate; never CV at the aggregated level alone.

R Packages Used

`forecast::tsCV()` for expanding-window time-series CV; `fable::stretch_tsibble()` and `accuracy()` for tidyverts CV workflows; `tibbletime::rollify()` for general rolling functions; `slider::slide_dbl()` for sliding-window operations.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Time series basics